

Modelo de arquitetura de aplicações em camadas baseado em padrões de projeto

Regis Pires Magalhães¹

Resumo: Este artigo propõe uma arquitetura que divide a aplicação em um conjunto de camadas que facilitem a divisão do trabalho e a flexibilidade de mudanças na aplicação, permitindo desde a fácil integração com as mais diversas tecnologias de persistência, até a apresentação utilizando variadas opções de interface com o usuário. O modelo baseia-se na busca de flexibilidade para o desenvolvimento de aplicações que sejam fáceis de manter e que possuam um prolongado tempo de vida, adaptando-se ao acoplamento a novas tecnologias. A divisão em camadas do modelo abordado no artigo está embasada em padrões de projeto que são melhores soluções práticas para problemas comuns recorrentes. Cada padrão de projeto descreve um problema que ocorre repetidamente, e, então, descreve o núcleo da solução para o problema, de modo tal que se possa usar eficazmente a mesma solução inúmeras vezes.

Palavras-chave: arquitetura de aplicações; padrões de projeto; divisão em camadas.

Abstract: This article presents an architecture that divides an application in a set of tiers in order to make changes related to division of work and application development easier to be done. This allows an easy integration from different persistence technologies to distinct user interface options. The main goal of this model is flexibility in application development which must be easy to maintain and have a long period of life. The model must fit easily to new technologies. The tier separation relies on design patterns, which are the best practical solutions to recurring problems. Each design pattern documents a problem that occurs repeatedly and shows the solution to that problem, in such a way that it can be reused over time.

Keywords: application architecture; design patterns; tier separation.

1. Introdução

Temos observado ao longo do tempo muitas aplicações empresariais complexas funcionando para atender às diferentes necessidades das organizações. Essas aplicações desempenham bem suas funções, mas quando alguma tecnologia precisa ser substituída, como o sistema gerenciador de banco de dados ou a interface do usuário, por exemplo, a aplicação praticamente precisa ser totalmente reimplementada. Esse modelo também conduz ao desenvolvimento de aplicações cujos membros da equipe possam implementar desde a interface do usuário até a persistência dos dados. A equipe inteira fica constituída de “clínicos gerais” e carente de especialistas em determinadas áreas de atuação. Isso torna a aplicação um bloco monolítico sem divisão de tarefas dentro de um determinado serviço da aplicação. Todos tentam dominar todas as etapas do desenvolvimento e a aplicação termina ficando superficial em seus detalhes.

O artigo enfoca um modelo de arquitetura de aplicação que minimize os problemas descritos acima, dando maior flexibilidade para a substituição de tecnologias com ampla reutilização de código. Para isso, a aplicação é dividida em camadas responsáveis por tarefas específicas no âmbito de cada serviço.

2. Uso de padrões e divisão em camadas

Padrões permitem a documentação de um problema comum recorrente e sua solução em um determinado contexto. Um dos elementos chave da sentença anterior é a palavra recorrente, desde que o objetivo principal dos padrões é promover seu reuso ao longo do tempo.

Os padrões tiveram início nos anos 70 quando Christopher Alexander (apud ALUR et al, 2001, p. 7) escreveu alguns livros documentando padrões nas áreas de engenharia civil e arquitetura. Posteriormente a comunidade de software adotou a idéia de padrões baseados nos trabalhos de Christopher Alexander. Entretanto, os padrões ganharam popularidade com a publicação do livro *Design Patterns: Elements of Reusable Object-Oriented Software* (GAMMA et al, 1995). Os padrões que eles descreveram não foram criados por eles, mas foram catalogados pelos autores a partir de uma série de tarefas recorrentes identificadas em vários projetos.

De acordo com Christopher Alexander cada padrão é uma regra de três partes, que expressa uma relação entre um determinado contexto, um problema e uma solução (apud ALUR et al, 2001, p. 8).

¹ Aluno da Especialização em Tecnologias para WEB da Universidade Federal do Piauí. 2003.

Várias categorias foram sugeridas para classificar os padrões relacionados a software. As mais comuns são: padrões de projeto, padrões de arquitetura, padrões de análise, padrões de criação, padrões de estruturação e padrões comportamentais (ALUR et al, 2001, p. 9). Mesmo nessas categorias há vários níveis de abstração e de classificação ortogonal. Assim, não há uma única maneira documentar essas idéias.

Segundo ALUR et al (2001, p. 128) a visão dos padrões pode ser delineada em termos de camadas. Uma camada define uma separação de atribuições no sistema. Cada camada possui uma responsabilidade única no sistema, estando logicamente separada das demais camadas. Além disso, está livremente interligada à camada adjacente. ALUR et al (2001, p. 129) propõem, então, cinco camadas que estabelecem uma separação lógica de interesses. As camadas são descritas a seguir.

Camada cliente

Camada que representa todos os dispositivos ou clientes acessando a aplicação. Um cliente pode ser um navegador, uma aplicação *Java*, um *applet*, um dispositivo móvel, uma aplicação de rede ou alguma outro tipo de tecnologia que possua componentes de interação com o usuário. Pode inclusive ser um processo em lote.

Camada de apresentação

Essa camada encapsula toda a lógica de apresentação usada para servir os clientes. A camada de apresentação intercepta as requisições dos clientes, conduz o gerenciamento de sessão e o acesso a serviços de negócio, constrói as respostas e envia as respostas aos clientes. *Servlets* e *JSP* (*JavaServer Pages*) residem nessa camada. Observe que eles não são elementos da interface com o usuário, mas produzem tais elementos. A produção de elementos de interface não ocorre, no entanto, em aplicações *Java* e *applets* que não utilizam *servlets* e *JSP*. Em tais aplicações, a camada de apresentação estaria ausente e a camada cliente faria uso direto dos serviços da camada de negócio.

Camada de negócio

Essa camada provê serviços de negócio requeridos pelas aplicações clientes. Contém os dados de negócio e a lógica de negócio. A lógica de negócio está voltada para o fluxo de trabalho da aplicação. É independente da interface com o usuário e dos dados persistentes. A camada cliente/apresentação torna-se, assim, bastante leve, somente contendo o código necessário para chamar as interfaces de negócio e para exibir ou gerar os resultados. As regras de negócio são mais concisamente expressas e de fácil manutenção se as noções essenciais de gerenciamento de persistência forem movidas para outra camada.

Camada de integração

Essa camada é responsável pela comunicação com recursos externos como sistemas de armazenamento de dados e sistemas legado. A camada de negócio utiliza a camada de integração sempre que um objeto de negócio requer dados ou serviços que residem na camada de recurso. Os componentes dessa camada podem usar *JDBC* (*Java Database Connectivity*), tecnologia de conector *J2EE* (*Java 2 Enterprise Edition*), ferramentas de mapeamento objeto-relacional, *JDO* (*Java Data Objects*) ou qualquer outra tecnologia que possibilite acesso à camada de recurso. Deve possuir ainda as seguintes características:

- Não requer regras de negócio da aplicação. Somente deve tratar de detalhes de uma tarefa se sua execução for permitida pela lógica de negócio.
- Não precisa tratar de questões relativas à segurança;
- Pode preocupar-se com preservação da integridade dos dados;
- Requer o conhecimento da tecnologia de persistência para poder implementar operações.

Camada de recurso

Essa camada contém recursos externos à aplicação como sistemas gerenciadores de bancos de dados, *mainframes* e sistemas legado.

3. Características do modelo de arquitetura proposto

Agora mostraremos algumas características relevantes do modelo. Elas nortearam a adoção dos padrões que serão analisados no próximo item.

A divisão em camadas propicia suporte a diversas tecnologias de cliente, como aplicações web, interfaces gráficas usando *Java Swing*, outros sistemas de janela ou ainda *applets*. Os clientes implementados são clientes leves, ou seja, com pouca exigência de recursos da máquina onde vão ser executados. A camada cliente/apresentação pode, assim, ser modificada sem nenhuma alteração nos objetos de negócio. Além disso, os objetos de negócio podem ser testados sem referência à camada de apresentação. Para aplicações web, cujo visual normalmente sofre muitas modificações ao longo do tempo, essa abordagem torna-se particularmente vantajosa, permitindo alterações em um código mais estreitamente relacionado ao âmbito das modificações.

O modelo também permite uma melhor definição e distribuição de tarefas pela equipe de trabalho. Assim, membros da equipe podem tornar-se especialistas em determinadas camadas, garantindo maior profissionalismo e eficiência ao lidar com cada camada da aplicação. A divisão em camadas favorece a divisão de responsabilidades.

Permite ainda a portabilidade entre bancos de dados e a possibilidade de acessos usando a *API* (*Application Programming Interface*) padrão da linguagem Java (*JDBC*), alguma ferramenta de mapeamento objeto-relacional ou ainda tecnologias que permitam o acesso a bancos XML (*Extensible Markup Language*), orientado a objeto ou texto. Independentemente da estratégia de acesso a dados utilizada, uma camada de abstração é usada para separar a camada de negócio dos detalhes de acesso a dados da camada de integração.

A arquitetura enfocada até aqui ainda possibilita a fácil substituição do modelo cliente-servidor tradicional em que o cliente integra as quatro primeiras camadas, fazendo uso da camada de recurso por um modelo cliente-servidor multi-camada que envolve um cliente leve (camada cliente) acessando um servidor de aplicações *J2EE* ou simplesmente um contêiner web (camadas de apresentação, negócio e integração) que por sua vez faz uso da camada de recurso. Essas duas abordagens aparentemente contraditórias podem ser facilmente alternadas ou usadas simultaneamente com a utilização do modelo de divisão em camadas proposto neste artigo.

O modelo também prioriza o uso de interfaces, ao invés de classes concretas para deixar a aplicação mais flexível a futuras alterações no modelo de negócio ou nas tecnologias usadas. O primeiro princípio de projeto orientado a objeto reutilizável proposto por GAMMA et al (1995) é: “ programe para interfaces e não para implementações”. Isso separa as interfaces de suas implementações, promovendo maior flexibilidade. O máximo de flexibilidade é atingido ao se declarar atributos e parâmetros com o tipo mais genérico possível. O uso de interfaces aumenta um pouco a complexidade e traz uma pequena perda em performance. As vantagens, por outro lado, superam as desvantagens desde que o uso de interfaces possibilita alterações na implementação da classe de um objeto sem afetar o código de chamada, permitindo a parametrização de qualquer parte da aplicação sem afetar outros componentes. Também é possível prover testes e protótipos de implementações, facilitando os testes de outras classes e permitindo múltiplas equipes trabalharem em paralelo, após terem feito um acordo sobre a interface.

JOHNSON (2002, p. 130) propõe que camadas de abstração devem ser criadas ao nível de exceções. Assim, uma exceção ocorrida em uma camada deve lançar uma exceção específica daquela camada, encapsulando as exceções de outras camadas. Assim, uma *java.rmi.RemoteException* seria encapsulada por uma exceção relacionada à interface com o usuário e não uma exceção que praticamente forçaria a interface gráfica a lidar com o mecanismo de chamadas remotas. A tarefa de uma exceção deve estar bem clara em seu nome e não na mensagem que ela carrega, desde que as mensagens não favorecem um tratamento de erro tão direto como a simples captura (*catch*). Quando diferentes problemas requerem ações distintas, exceções correspondentes aos problemas devem ser criadas.

Um item que geralmente causa intrigantes dúvidas nos desenvolvedores diz respeito ao local onde deve ser realizada a validação de dados. Essa validação pode ocorrer em três camadas de uma aplicação:

- Camada cliente, através do uso de *JavaScript* em aplicações web, por exemplo. Nessa abordagem a validade precede a submissão do formulário, diminuindo o tráfego da rede e respondendo mais prontamente o cliente.
- Camada de apresentação. Uma classe de controle ou auxiliar valida os dados após a submissão do formulário e redireciona o usuário para o formulário se os dados forem inválidos, não utilizando nenhum objeto de negócio.
- Camada de negócio. Validação realizada nos objetos de negócio.

Para JOHNSON (2002, p. 496) a escolha da camada ou camadas para realização da validação normalmente é função do tipo de validação empregada. Geralmente as validações são sintáticas ou semânticas. Validações sintáticas estão relacionadas a checar a presença, tamanho ou formato dos dados. Isso não é lógica de negócio e pode ser realizado em uma camada diferente da camada de negócio. As validações semânticas, por outro lado, podem envolver lógica de negócio e até mesmo acesso a dados persistentes.

Qualquer camada que seja eleita para validar os dados tem vantagens e desvantagens. O uso de *JavaScript* reduz a carga no servidor e diminui o tempo de resposta. Por outro lado, um extenso código *JavaScript* dificulta a manutenção. Além disso, *JavaScript* pode ocasionar comportamentos diferentes de acordo com o navegador que esteja sendo usado. Na prática, validações *JavaScript* devem ser feitas, mas não de forma isolada, evitando assim que o usuário manipule as requisições de modo a obter efeitos desastrosos à aplicação.

A validação na camada de apresentação tem a grande desvantagem de atrelar a lógica de validação que pode ser lógica de negócio à API de servlets e talvez à framework web. O certo é que para maximizar o potencial de reuso dos códigos de validação, a validação preferivelmente deve ser realizada nos objetos de negócio. Isso amplia o reuso de objetos de validação por variados tipos de cliente.

4. Padrões adotados nas camadas do modelo de arquitetura proposto

4.1. Camada cliente / apresentação

Padrão de arquitetura Modelo-Visão-Controlador (MVC)

A camada cliente/apresentação pode ser subdividida em três componentes seguindo o amplamente difundido padrão de arquitetura MVC – Modelo-Visão-Controlador. A arquitetura MVC foi inicialmente documentada para interfaces com o usuário na linguagem *Smalltalk* (HUSTED, 2003, p. 41). Ele é base do pacote gráfico *Swing* da linguagem *Java* e da amplamente difundida *framework* web *Struts*. A arquitetura MVC divide os componentes necessários para construir uma interface em três tipos de objetos, garantindo uma clara separação de papéis:

- um modelo de dados (*model*) ou objeto de aplicação que não contém código de apresentação;
- um objeto de visão (*view*), responsável pela exibição do modelo de dados;
- um objeto de controle (*controller*) que reage às entradas do usuário, atualizando o modelo de dados.

O modelo de dados (*model*) proposto neste artigo é um *JavaBean* que expõe dados. Um objeto modelo não deve saber como recuperar dados dos objetos de negócio. Ao invés disso, ele expõe propriedades que permite alterações em seu estado pelo controlador. Em nosso caso eles são objetos de transferência de dados (DTO – *Data Transfer Objects*, também chamados VO – *Value Objects*), representando o resultado de uma operação de negócio. O modelo não deve ser dependente da API de servlet nem de uma framework web.

Uma visão é um componente usado para exibir dados de um modelo. Uma visão não deveria realizar lógica de negócio ou obter dados além dos disponíveis no modelo. Assim, múltiplas visões podem caracterizar o mesmo modelo, sem interferir no comportamento da aplicação. A visão é constituída por *JSP*, *XSLT*, *templates Velocity* e *WebMacro*, *XMLC* ou outra tecnologia que permita a geração ou visualização da interface com o usuário. A visão não deve fazer recuperação de dados, como por exemplo, consultas SQL. Ao invés disso, deve meramente exibir dados de um ou mais modelos. A visão pode realizar lógica de apresentação que é bem diferente de lógica de controle e de negócio. Um exemplo disso é a visualização de valores numéricos negativos na cor vermelha ou a exibição de informação adicional caso o modelo contenha um valor opcional.

O controlador é uma classe Java que trata requisições, interage com objetos de negócio, constrói modelos e direciona cada requisição à visão adequada. Tal controlador não implementa lógica de negócio, que não é função da camada de apresentação. Uma implementação típica envolve um *servlet* controlador, que age como único ponto de entrada em uma aplicação ou em um subconjunto de *URLs* (*Uniform Resource Locators*) da aplicação. Esse ponto de entrada seleciona um dos vários controladores de requisição, responsáveis pelo tratamento da requisição. Os mapeamentos são normalmente realizados em arquivos de configuração fora do código Java. Na framework *Struts* os controladores de requisições são chamados de *actions*. Eles não produzem saída, mas processam requisições, iniciam operações de negócio e, opcionalmente, manipulam estado de sessão e aplicação, além de redirecionarem requisições à visão adequada.

Sistemas construídos com uso da abordagem MVC tendem a ser flexíveis e extensíveis. Desde que a apresentação fica separada da lógica, alterações significativas na apresentação não afetam seu comportamento, sendo inclusive possível substituir uma tecnologia de visão por outra.

Consideremos agora o fluxo de controle de uma implementação MVC simplificada:

- Um *servlet* de controle recebe uma requisição. Há um mapeamento da URL requisitante para o *servlet* de controle em um descritor da aplicação em formato *XML*.
- O *servlet* de controle examina os parâmetros da requisição. Analisa quais parâmetros são obrigatórios. Se um deles faltar ou estiver com tipo ou tamanho inadequados, o usuário deve ser redirecionado à visão `página_inicial.jsp`.
- Se os dados forem válidos, o *servlet* de controle invoca o método de negócio adequado à operação.

- Caso a operação seja bem sucedida, o *servlet* recebe o resultado (modelo) e o adiciona à `HttpServletRequest` como um atributo, redirecionando a requisição para a visão `exibir_resultado.jsp`. Caso nenhum resultado seja encontrado, o *servlet* adiciona uma exceção contendo essa informação à `HttpServletRequest` como atributo e redireciona a requisição para a visão `nova_tentativa.jsp`.

4.2. Camada de negócio

4.2.1. Padrão de projeto BO (Business Object)

Um objeto de negócio é um objeto que implementa a lógica de negócio e/ou dados de negócio. Na maioria dos casos um objeto de negócio é um objeto Java comum que provê alguns serviços (ALUR et al, 2001, p. 133). Os objetos de negócio recebem os dados da camada de apresentação e executam as operações e validações necessárias para enviá-los à camada de integração. Da mesma forma, extrai os dados da camada de integração e de acordo com as regras de negócio da aplicação os encaminha para a camada de apresentação a fim de que possam ser formatados e exibidos. Objetos de negócio geralmente são responsáveis pelo controle de transações, segurança e *logs* de aplicações.

Comumente uma interface de negócio é criada para definir as operações que permitem a manipulação da camada de integração. A camada de apresentação acessa os objetos de negócio através de suas interfaces de negócio, separando a funcionalidade da implementação do código. A implementação das classes de negócio normalmente delega a execução de suas tarefas à camada de integração.

A nomenclatura que se convencionou usar no modelo proposto neste artigo acrescenta o sufixo BO para definição de interfaces de negócio e o sufixo BOImpl para especificar classes que implementam as interfaces de negócio.

4.3 Camada de integração

4.3.1. Padrão de projeto DAO (*Data Access Object*)

A melhor maneira de garantir a portabilidade na camada de integração é usar uma camada de abstração sobre implementações específicas da plataforma destino. A camada de abstração consiste de interfaces Java comuns, de modo que se possa usar qualquer estratégia de acesso a dados, inclusive simultaneamente, mesmo usando características nativas das tecnologias usadas da camada de recurso sem, contudo, comprometer a portabilidade.

Há várias opções para implementação da lógica de persistência. Algumas opções são: *JDBC*, *Entity Beans* (plataforma J2EE), *Java Data Objects (JDO)*, *frameworks* de persistência (*TopLink*, *Hibernate* ou outra), *Java Connector Architecture (JCA)* e *Stored Procedures*. Outro fato a ser levado em conta é o tipo de banco de dados que será usado. Pode-se optar por bancos relacionais, objeto-relacionais, orientados a objeto, XML, legado, texto ou outros. A adoção de um tipo específico de banco de dados pode ter influência na escolha da tecnologia de persistência que fará uso desse banco. Além disso, novas tecnologias de bancos de dados e de manipulação aparecerão. É evidente que estabelecer uma ligação direta entre a lógica de negócio e a tecnologia de persistência prende a aplicação a uma determinada tecnologia de persistência. A solução para obter maior flexibilidade de escolha, uso simultâneo ou alternado de diferentes tecnologias e reaproveitamento de código da lógica de negócio em caso de futuras alterações das tecnologias de acesso a dados, é criar uma interface que possa abstrair a lógica de persistência e, conseqüentemente, o tipo de banco de dados. Essa interface define a camada de abstração de acesso a dados definida pelo padrão de projeto Objeto de Acesso a Dados (*Data Access Object – DAO*).

Ao estabelecer uma distinção entre lógica de negócio e lógica de persistência, pode-se separá-los pela definição de uma interface de persistência que os componentes da lógica de negócio pode usar. O padrão de projeto *Data Access Object (DAO)* usa uma camada de abstração de interfaces Java comuns entre os componentes da lógica de negócio e da lógica de persistência. As implementações dessas interfaces tratam da lógica de persistência. O padrão de projeto DAO é descrito em detalhes por ALUR et al (2001, p. 290), embora seja somente um caso particular do padrão de projeto *Strategy* de GAMA et al (1995). O padrão de projeto DAO também pode trabalhar dentro ou fora de um contêiner EJB (Enterprise JavaBeans), ampliando ainda mais a flexibilidade desta arquitetura.

De acordo com JOHNSON (2002, p. 26), o uso da *API JDBC* somente deve ser estimulado quando não há um mapeamento objeto-relacional natural ou quando esse mapeamento não garante uma eficiência aceitável. Isso ocorre porque o *JDBC* é de baixo nível e bastante trabalhoso. A prática demonstra que somente uma pequena parte de um código que faz uso de *JDBC* é destinada efetivamente ao comando que vai ser executado na camada de recurso. Além disso, o *JDBC* não permite parâmetros nomeados, exigindo que tenhamos de nos preocupar demasiadamente com a contagem de parâmetros de um comando SQL complexo. Essa contagem se torna ainda mais enfadonha quando um comando SQL com várias referências a um mesmo parâmetro é usado. Outro caso em que vale a pena usar *JDBC* é quando se quer fazer uso de recursos avançados do sistema gerenciador de banco de dados, como procedimentos armazenados (*stored procedures*). Abaixo estão relacionadas algumas características do padrão de projeto DAO:

- Não está necessariamente baseado no mapeamento objeto-relacional, embora possa ter uma implementação baseada nesse tipo de mapeamento;
- É uma abordagem leve, baseada em bons princípios de projeto orientado a objeto.
- Usa interfaces *Java* comuns ao invés de uma infra-estrutura especial como a dos *entity beans*.
- Não objetiva o tratamento de transações ou segurança, deixando isso a cargo nos componentes da lógica de negócio.
- Funciona com qualquer tecnologia de persistência subjacente.

O uso de diferentes tecnologias de acesso a dados não necessita nenhuma alteração na lógica de negócio para acomodar diferentes estratégias de acesso a dados, desde que os objetos de negócio acessem o DAO através de uma interface fixa. O padrão DAO consiste, então, de uma interface que abstrai a tecnologia de armazenamento e de classes concretas que implementam essa interface fixa.

4.4. Comunicação entre camadas

4.4.1. Padrão de projeto *DTO* (*Data Transfer Object*) ou *VO* (*Value Object*)

Não basta dividir a aplicação em camadas. Essas camadas precisam comunicar-se entre si. A camada de apresentação utiliza os métodos da camada de negócio que por sua vez, usa serviços providos pela camada de integração. Em certos casos, os métodos chamados necessitariam receber um grande número de parâmetros. Outras vezes precisariam retornar vários valores, quando em *Java* um único valor é permitido para retorno. Uma solução adotada para resolver esse problema é o uso de um objeto de transferência de dados ou valores entre camadas, chamado *Value Object* (*VO*) ou *Data Transfer Object* (*DTO*). Tal objeto é um *JavaBean* serializável composto somente de atributos e métodos de leitura e gravação dos atributos, também chamados métodos *getter* e *setter*.

Um *DTO* é um objeto *Java* serializável que pode passar eficientemente pela rede e que somente contém os dados necessários do contexto em que é posto em uso. Os objetos de transferência de dados também são chamados de objetos de valor (*value objects*). A estratégia do padrão de projeto *DTO* consolida múltiplos valores de dados em um único objeto serializável para minimizar o número de chamadas remotas requeridas.

5. Conclusão

O desenvolvimento de aplicações divididas nas camadas cliente, apresentação, negócio, integração e recurso aumenta a complexidade da aplicação, ao mesmo tempo em que favorece a implementação de aplicações mais bem elaboradas, robustas, com mais flexibilidade a mudanças e maior tempo de vida, especialmente quando cada camada é desenvolvida com base nas melhores soluções a problemas comuns recorrentes descritos em padrões de arquitetura e projeto.

6. Bibliografia

ALUR, Deepak; CRUPI, John; MALKS, Dan. **Core J2EE Patterns: Best Practices and Design Strategies**. Palo Alto: Sun Microsystems Press, 2001. 460 p.

BODOFF, Stephanie *et al.* **The J2EE Tutorial**. Boston: Addison-Wesley, 2002. 491 p.

GAMMA, Erich; HELM, Richard; JOHNSON, Ralph; VLISSIDES, John. **Design Patterns: Elements of Reusable Object-Oriented Software**. Boston: Addison-Wesley, 1995.

HUSTED, Ted; DUMOULIN, Cedric; FRANCISCUS, George; WINTERFELDT, David. **Struts in Action**. Greenwich: Manning, 2003. 630 p.

JOHNSON, Rod. **Expert One-on-One J2EE Design and Development**. Birmingham: Wrox Press, 2002. 750p.

MARINESCU, Floyd. **EJB Design Patterns: Advanced Patterns, Processes and Idioms**. New York: Wiley Computer Publishing, 2002. 259 p.

SINGH, Inderjeet *et al.* **Designing Enterprise Applications with the J2EE Platform**. 2. ed. Boston: Addison-Wesley, 2002. 418 p.